

Assignment Code: DS-AG-028

NLP Introduction & Text Processing | Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: What is Computational Linguistics and how does it relate to NLP?

Answer:

Computational linguistics is the scientific field that studies how computers can model and process human language using mathematical, algorithmic, and linguistic principles. It focuses on the structure of language—syntax, semantics, phonology—and how to formalize them.

NLP is the engineering side built on top of this science. Computational linguistics provides the theories; NLP turns them into practical systems such as chatbots, machine translation, and sentiment analysis.

Question 2: Briefly describe the historical evolution of Natural Language Processing.

Answer:

NLP evolved through four major phases:

1950s–1960s: Rule-based systems

Language understanding relied on manually written grammar rules and symbolic logic.

1970s–1980s: Knowledge-based systems

Systems used linguistic databases, ontologies, and expert systems.

1990s–2010: Statistical NLP

Machine learning models (HMMs, n-grams, CRFs) replaced hand-crafted rules.

2011–present: Neural NLP / Deep Learning

Word embeddings, RNNs, LSTMs → Transformers → LLMs dominated the field.

Question 3: List and explain three major use cases of NLP in today's tech industry.

Answer:

Customer support automation – Chatbots and virtual assistants reduce manpower and handle large volumes of queries.

Sentiment analysis – Companies detect opinions, satisfaction levels, and product sentiment from reviews or social media.

Machine translation – Tools like Google Translate use neural models to convert text across languages in real time.

Question 4: What is text normalization and why is it essential in text processing tasks?

Answer:

Text normalization is the process of converting raw text into a consistent, machine-friendly format. It includes lowercasing, removing punctuation, expanding contractions, correcting spellings, etc.

It's essential because models perform poorly on inconsistent or noisy text. Normalization ensures uniform input so tokenization, feature extraction, and ML algorithms work correctly.

Question 5: Compare and contrast stemming and lemmatization with suitable examples.

Answer:

Stemming

- **Cuts words using crude chopping rules.**
- **Ignores grammar and context.**
- **Output may not be a valid dictionary word.**
Example:
- **“running” → “run” (ok)**

- “studies” → “studi” (incorrect word)

Lemmatization

- Uses vocabulary + morphological analysis.
- Returns meaningful dictionary words.
Example:
- “running” → “run”
- “studies” → “study”

Question 6: Write a Python program that uses regular expressions (regex) to extract all email addresses from the following block of text:

“Hello team, please contact us at support@xyz.com for technical issues, or reach out to our HR at hr@xyz.com. You can also connect with John at john.doe@xyz.org and jenny via jenny_clarke126@mail.co.us. For partnership inquiries, email partners@xyz.biz.”

(Include your Python code and output in the code box below.)

Answer:

```
import re
```

```
text = """Hello team, please contact us at support@xyz.com for technical issues, or reach out to  
our HR at hr@xyz.com. You can also connect with John at john.doe@xyz.org and jenny  
via jenny_clarke126@mail.co.us. For partnership inquiries, email partners@xyz.biz."""
```

```
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", text)
```

```
print(emails)
```

```
output
```

```
['support@xyz.com', 'hr@xyz.com', 'john.doe@xyz.org',  
'jenny_clarke126@mail.co.us', 'partners@xyz.biz']
```

Question 7: Given the sample paragraph below, perform string tokenization and frequency distribution using Python and NLTK:

“Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical.”

(Include your Python code and output in the code box below.)

Answer:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist
```

```
paragraph = """Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical."""
```

```
tokens = word_tokenize(paragraph.lower())
freq = FreqDist(tokens)
```

```
print(tokens)
print(freq.most_common(10))
```

output sample

```
['natural', 'language', 'processing', ...]
[('nlp', 3), ('language', 2), ('and', 2), ('a', 2), ('of', 2), ...]
```

Question 8: Create a custom annotator using spaCy or NLTK that identifies and labels proper nouns in a given text.

(Include your Python code and output in the code box below.)

Answer:

```
import spacy
from spacy.tokens import Span

nlp = spacy.load("en_core_web_sm")

text = "John works at Microsoft and lives in New York City."
doc = nlp(text)

proper_nouns = []
for token in doc:
    if token.pos_ == "PROPN":
        proper_nouns.append((token.text, token.pos_))

print(proper_nouns)
```

Question 9: Using Genism, demonstrate how to train a simple Word2Vec model on the following dataset consisting of example sentences:

dataset = [

"Natural language processing enables computers to understand human language",

"Word embeddings are a type of word representation that allows words with similar meaning to have similar representation",

"Word2Vec is a popular word embedding technique used in many NLP applications",

"Text preprocessing is a critical step before training word embeddings",

"Tokenization and normalization help clean raw text for modeling"

]

Write code that tokenizes the dataset, preprocesses it, and trains a Word2Vec model using Gensim.

(Include your Python code and output in the code box below.)

Answer:

```
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess

dataset = [
    "Natural language processing enables computers to understand human language",
    "Word embeddings are a type of word representation that allows words with similar meaning to have similar representation",
    "Word2Vec is a popular word embedding technique used in many NLP applications",
    "Text preprocessing is a critical step before training word embeddings",
    "Tokenization and normalization help clean raw text for modeling"
]

# Tokenize & preprocess
tokens = [simple_preprocess(sentence) for sentence in dataset]

# Train model
model = Word2Vec(sentences=tokens, vector_size=100, window=5, min_count=1, workers=4)

print(model.wv.most_similar("language"))
```

Question 10: Imagine you are a data scientist at a fintech startup. You've been tasked with analyzing customer feedback. Outline the steps you would take to clean, process, and extract useful insights using NLP techniques from thousands of customer reviews.

(Include your Python code and output in the code box below.)

Answer:

Process Outline

1. Load data – Read thousands of reviews from CSV.
2. Clean text – Remove URLs, HTML, emojis, special characters.
3. Normalize – Lowercase, remove stopwords, lemmatize.
4. Tokenize – Split into words or sentences.
5. Vectorize – TF-IDF, embeddings, or transformer encodings.
6. Sentiment analysis – Predict polarity, classify feedback.
7. Topic modeling – Use LDA or clustering to discover major complaint themes.
8. Summaries – Generate concise summaries of common issues.

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
```

```
# Example reviews
```

```
reviews = pd.Series([
    "The app keeps crashing during payment.",
    "Customer support was very helpful.",
    "Payment gateway failed multiple times.",
    "UI is clean but loading is slow."
])
```

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()
```

```
def clean_text(t):  
    t = re.sub(r"http\S+|[\^a-zA-Z ]", " ", t)  
    t = t.lower()  
    tokens = [lemmatizer.lemmatize(w) for w in t.split() if w not in stop_words]  
    return " ".join(tokens)  
  
cleaned = reviews.apply(clean_text)  
  
vectorizer = TfidfVectorizer()  
X = vectorizer.fit_transform(cleaned)  
  
kmeans = KMeans(n_clusters=2, n_init='auto')  
kmeans.fit(X)  
  
print("Cluster labels:", kmeans.labels_)
```